12. Consumer Setup - Part2

Saturday, 14 March 2026 9:25 PM

In Part-1, already seen how to setup basic consumer.



Till now in Basic Consumer setup, we have seen, how to create a Consumer and how it can read from a Topic.

But what if:

-> 1 consumer request to join more than 1 topic?
-> multiple consumers in a group request to join different topic?
-> multiple consumers in a group request to join same topic?

How to achieve this? And this is where things get interesting.

In Basic Consumer setup:

```
@Component
public class OrderEventListener {

@KafkaListener(topics="order-events")
public void consume (Order order) {
        //business logic
        System.out.println("Received:" + order.getOrderId());
  }
}
```

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

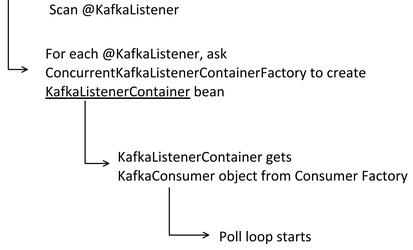
spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

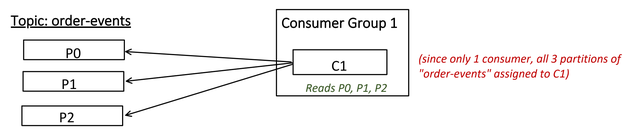
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

What we saw before is:

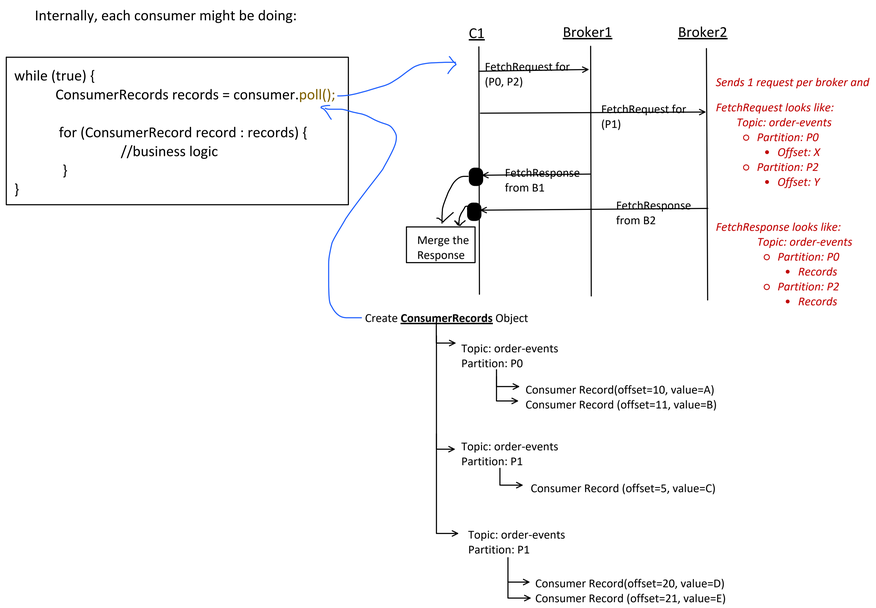


Let say: "order-events" topic has 3 partition (P0, P1 and P2), so with above Basic consumer setup, what we achieve is:

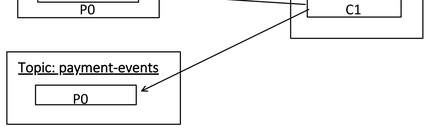


Partition	Leader Broker
P0	B1
P1	B2
P2	B1

Internally, each consumer might be doing:



1st: how to set up, 1 consumer request to join more than 1 topic?



Approach 1 : Manually taking care of Json to object mapping

application.properties

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

Now this consumer handles more than 1 topic, so I will handle the Deserialization like if its order-events -> convert it to Order object If its payment-events -> convert it to Payment object.

So, that's why here I am using StringDeserializer. As I need to control the deserialization.

```
@Component
public class OrderEventListener {

@Autowired
ObjectMapper objectMapper;

@KafkaListener(topics= {"order-events", "payment-events"})
public void consume (ConsumerRecord<String, String> record) {
        If("order-events".equals(record.topic())) {

                Order order = objectMapper.readValue(record.value(), Order.class);
                  // business logic for order
        }
        else if("payment-events".equals(record.topic())) {

                Payment payment = objectMapper.readValue(record.value(), Payment.class);
                // business logic for order
        }
    }
}
```

Like this we can provide the list of topics which a Consumer wanted to subscribe.

here in parameter, I accept the record as it is and Deserialization task I am taking care of.

Approach 2: AUTO Mapping Json to object conversion

But before that first understand the very important JSON DESERIALIZATION FLOW, where many of us get confused:

At PRODUCER side (during JSON Serialization):

```
@Service
public class OrderProducerService {

@Autowired
private KafkaTemplate<String, Order> kafkaTemplate;

public void sendWithKey(Order order) {

        String key = order.getOrderId();

        kafkaTemplate.send("order-events", key, order);

 }
}
```

application.properties

```
.
.
.
#--------------SERIALIZER------------
# Key needs to be String
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

# Value: JsonSerializer, auto-converts Java objects to JSON bytes
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer
spring.kafka.producer.properties.spring.json.add.type.headers=true
.
.
.
```

By-default this (json.add.type.headers) *is true only*

With this, when producer send the event:

```
kafkaTemplate.send("order-events", order);
```

It will also add _TypeId_ = com.eda.producer.model.Order in Event header

Type id value is hidden but now its added in event header

```
| offset: 0 CreateTime: 1773812798798 keySize: 3 valueSize: 112 sequence: 0 headerKeys: [__TypeId__] key: O-2 payload: {"orderId":
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTran
 240 magic: 2 compresscodec: snappy crc: 2988695115 isvalid: true
```

But, its not always 100% sure that Producer will add "_TypeID_" or not, even when its true (which is default)

Like till now, we have seen, headerKeys were empty , even though spring.kafka.producer.properties.spring.json.add.type.headers=true (Default)

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log \
  --files /tmp/broker1-logs/order-events-3/00000000000000000000.log
Dumping /tmp/broker1-logs/order-events-3/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1000 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optio
nalLong.empty position: 0 CreateTime: 1773574261402 size: 135 magic: 2 compresscodec: snappy crc: 2489348583 isvalid: true
| offset: 0 CreateTime: 1773574261402 keySize: 7 valueSize: 42 sequence: 0 headerKeys: [] key: O-32700 payload: {"orderId":"O-32700","productId":"PROD-3"}
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```

Why because we are using KafkaTemplate<String, Order> kafkaTemplate. Specific value type Order is used, so it did not add _TypeId_

```
@Service
public class OrderProducerService {

@Autowired
private KafkaTemplate<String, Order> kafkaTemplate;

public void sendWithKey(Order order) {

        String key = order.getOrderId();

        kafkaTemplate.send("order-events", key, order);

    }
}
```

But if I use KafkaTemplate<String, Object> kafkaTemplate.

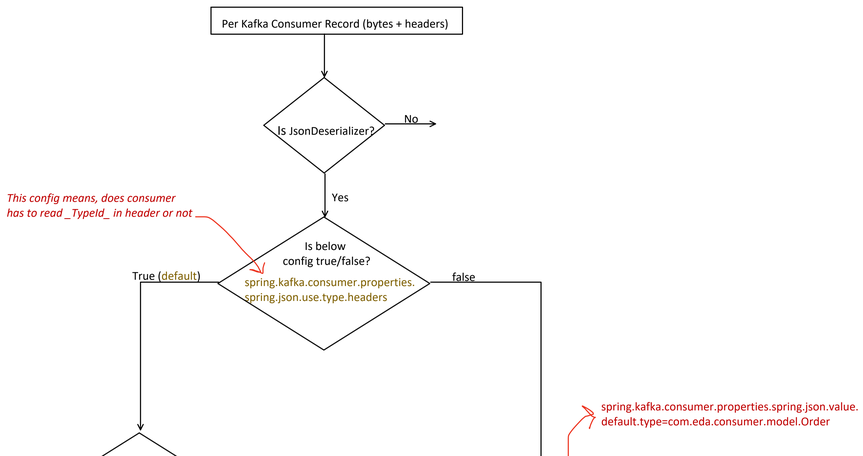
I used Object instead of Order, Now you will notice that Producer will add _TypeId_ in header.

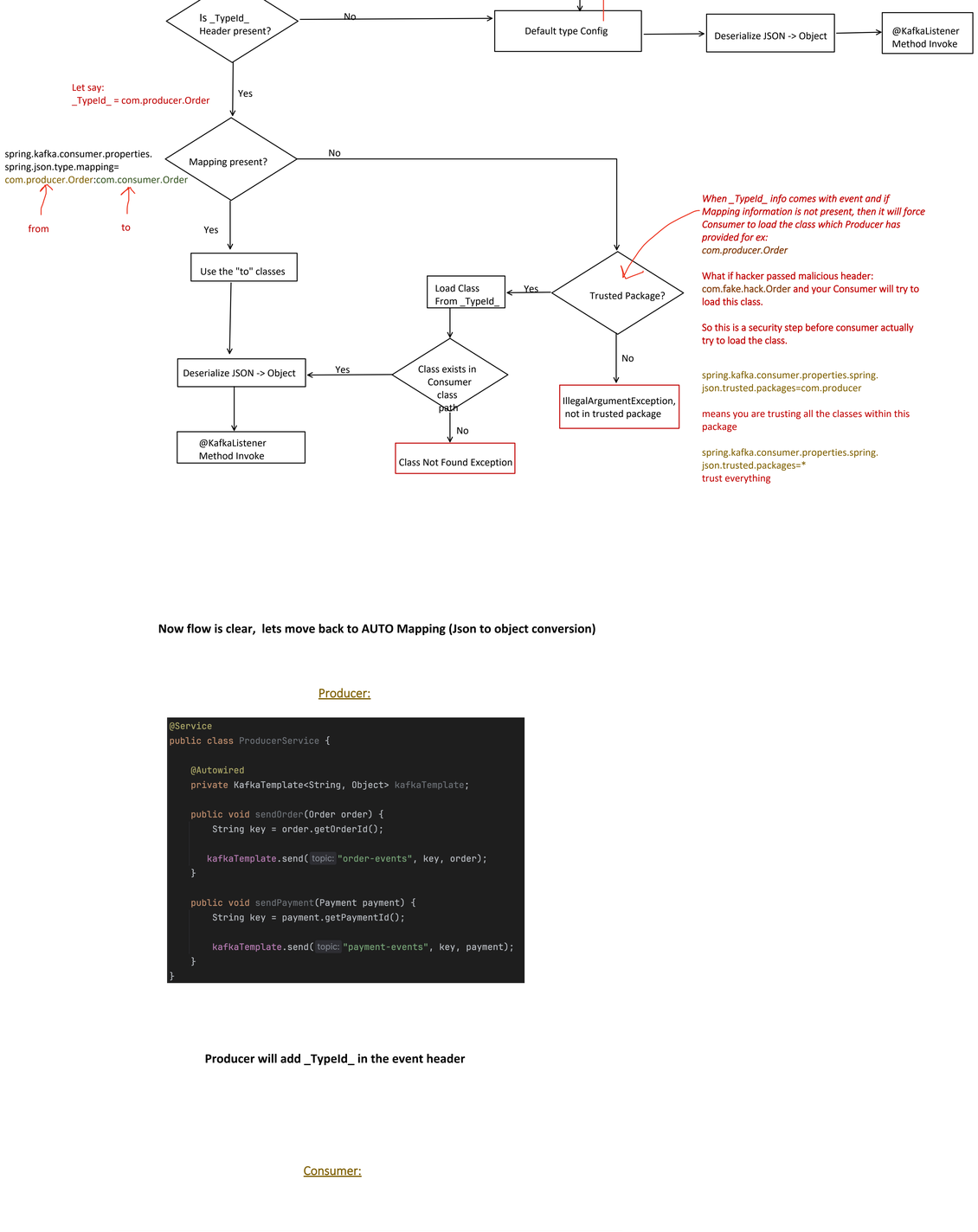
That's internal logic of Kafka producer client, but we have to understand that when Value uses JSON SERIALIZER then by-default below config is true:
spring.kafka.producer.properties.spring.json.add.type.headers=true

So producer will add _TypeId_ in event header.

TypeId = com.eda.producer.model.Order

At CONSUMER side (during JSON Deserialization):





Now flow is clear, lets move back to AUTO Mapping (Json to object conversion)

Producer:

```java
@Service
public class ProducerService {

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    public void sendOrder(Order order) {
        String key = order.getOrderId();

        kafkaTemplate.send( topic: "order-events", key, order);
    }

    public void sendPayment(Payment payment) {
        String key = payment.getPaymentId();

        kafkaTemplate.send( topic: "payment-events", key, payment);
    }
}
```

Producer will add _TypeId_ in the event header

Consumer:

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.type.mapping=
                com.eda.producer.model.Order:com.eda.consumer.model.Order,
                com.eda.producer.model.Payment:com.eda.consumer.model.Payment


spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

POJO Classes in Consumer

```java
package com.eda.consumer.model;

public class Order {

    private String orderId;
    private String customerId;
    private String productId;
    private Integer quantity;
    private Double totalAmount;
    private String status;

    //getters and setters
}
```

```java
package com.eda.consumer.model;
```

```java
@Component
public class OrderEventListener {

@KafkaListener(topics= {"order-events", "payment-events"})
public void consume (ConsumerRecord<String, String> record) {
        If("order-events".equals(record.topic())) {

                Order order = (Order) record.value();
                  // business logic for order
        }
        else if("payment-events".equals(record.topic())) {

                Payment payment = (Payment ) record.value();
                // business logic for order
        }
    }
}
```

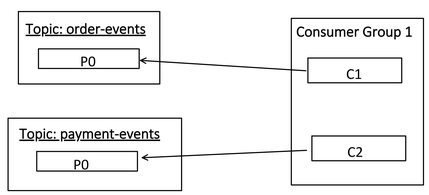
Now no manual mapping is required

```java
public class Payment {

    private String paymentId;
    private String customerId;
    private Double totalAmount;
    private String status;

   //getters and setters
}
```

2nd: how to set up, multiple consumers in a group request to join different topic?



Approach 1: 2 different consumer application

The most simple is, create 2 different Consumer application and whatever we have seen in basic consumer setup, that will work, as it creates 1 Consumer per application.

Approach 2: in 1 application itself we need 2 consumer

For each Kafka Listener,
1 KafkaConsumer is created

ConcurrentKafkaListenerContainerFactory to create
KafkaListenerContainer bean

KafkaListenerContainer gets
KafkaConsumer object from Consumer Factory

Poll loop starts

2 way:

TypeId + Mapping

Use Header = false

```
@Component
public class EventListener {

    @KafkaListener(topics="order-events")
    public void consumeOrder (Order order) {
        //business logic
        System.out.println("Received:" + order.getOrderId());
    }

    @KafkaListener(topics="payment-events")
    public void consumePayment (Payment payment) {
        //business logic
        System.out.println("Received:" + payment.getPaymentId());
    }
}
```

```
@Component
public class EventListener {

    @KafkaListener(topics="order-events")
    public void consumeOrder (Order order) {
        //business logic
        System.out.println("Received:" + order.getOrderId());
    }

    @KafkaListener(topics="payment-events")
    public void consumePayment (Payment payment) {
        //business logic
        System.out.println("Received:" + payment.getPaymentId());
    }
}
```

application.properties

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.type.mapping=
        com.eda.producer.model.Order:com.eda.consumer.model.Order,
        com.eda.producer.model.Payment:com.eda.consumer.model.Payment

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

application.properties

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.use.type.headers=false

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

This works fine and recommended.

This will not work, when payment event is sent:

- Consumer will consume it
- Use header = false
- Go to default type config, which is

  spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order

Exception will come:

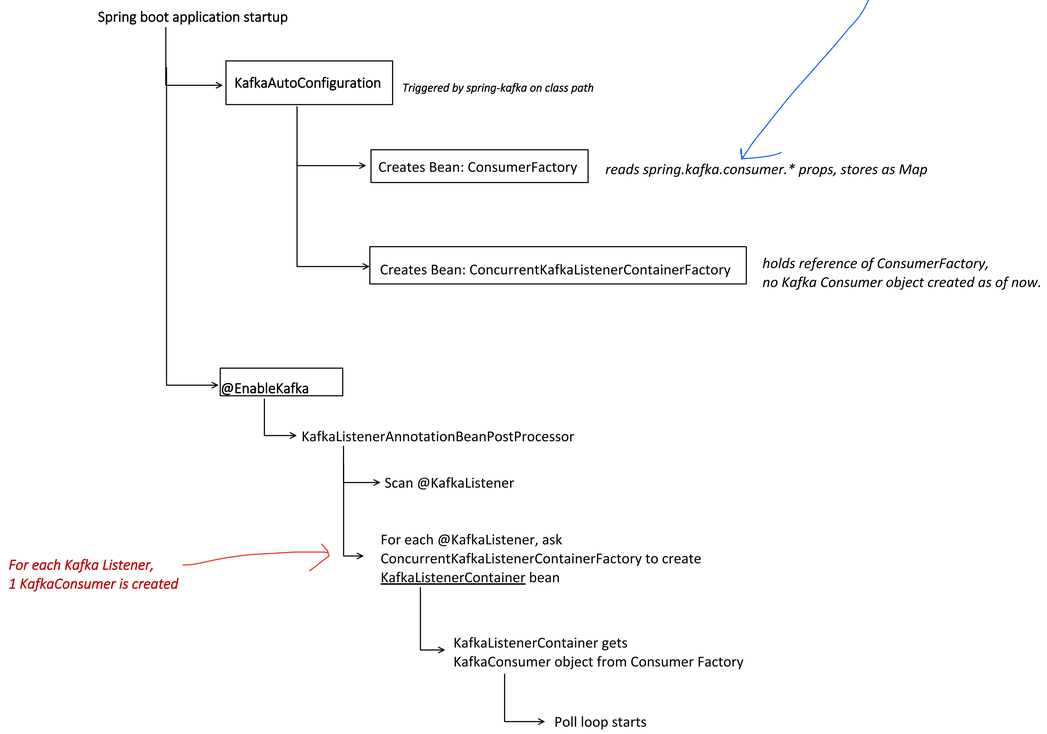
Caused by: org.springframework.messaging.converter.MessageConversionException: Cannot convert from [com.eda.consumer.model.Order] to [com.eda.consumer.model.Payment] for GenericMessage [payload=Order{orderId='null', customerId='CUST-2', productId='null', quantity=null, totalAmount=99.99, status='CREATED'}, headers={kafka_offset=0,

How to resolve this?

This issue comes because we have only 1 consumer Factory object which is created from properties defined in application.properties file.

So in this ***ConsumerFactory*** Object, "default Type" is what we set in application.properties i.e .

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order



And now Same **ConsumerFactory** object is used to create both KafkaConsumer:

- KafkaConsumer for Order events.
- KafkaConsumer for Payment event.

And since same ConsumerFactory object is used, both KafkaConsumer has "default Type" =
spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order

So if we get this, then solution is very simple, have different ConsumerFactory for each event type with default type is specific to each event.

Kafka Consumer Config

```
package com.eda.consumer.config;

import com.eda.consumer.model.Order;
import com.eda.consumer.model.Payment;
import org.springframework.boot.autoconfigure.kafka.KafkaProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.core.ConsumerFactory;
import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
import org.springframework.kafka.support.serializer.JsonDeserializer;
import java.util.Map;

@Configuration
    public class ConsumerConfig {

    @Bean
    public ConsumerFactory<String, Order> orderConsumerFactory(KafkaProperties props) {
            Map<String, Object> config = props.buildConsumerProperties();
            config.put(JsonDeserializer.VALUE_DEFAULT_TYPE, Order.class.getName());
            return new DefaultKafkaConsumerFactory<>(config);
    }

    @Bean
    public ConsumerFactory<String, Payment> paymentConsumerFactory(KafkaProperties props) {
        Map<String, Object> config = props.buildConsumerProperties();
        config.put(JsonDeserializer.VALUE_DEFAULT_TYPE, Payment.class.getName());
        return new DefaultKafkaConsumerFactory<>(config);
    }

    @Bean
```

```
    public ConcurrentKafkaListenerContainerFactory<String, Order> orderKafkaListenerFactory(ConsumerFactory<String, Order> orderConsumerFactory) {
        ConcurrentKafkaListenerContainerFactory<String, Order> factory = new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(orderConsumerFactory);
        return factory;
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, Payment> paymentKafkaListenerFactory(ConsumerFactory<String, Payment> paymentConsumerFactory) {
        ConcurrentKafkaListenerContainerFactory<String, Payment> factory = new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(paymentConsumerFactory);
        return factory;
    }
}
```

```
@Component
public class EventListener {

    @KafkaListener(topics="order-events", containerFactory="orderKafkaListenerFactory")
    public void consumeOrder (Order order) {
        //business logic
        System.out.println("Received:" + order.getOrderId());
     }

    @KafkaListener(topics="payment-events", containerFactory="paymentKafkaListenerFactory")
    public void consumePayment (Payment payment) {
        //business logic
        System.out.println("Received:" + payment.getPaymentId());
     }
}
```

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

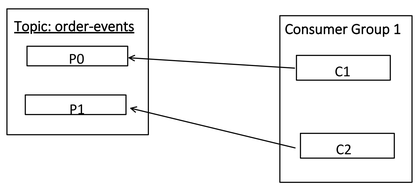
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.use.type.headers=false

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

3rd: how to set up, multiple consumers in a group request to join same topic?




```
@Component
public class EventListener {

    @KafkaListener(topics="order-events",)
    public void consumeOrder (Order order) {
```

```
        //business logic
        System.out.println("Received:" + order.getOrderId());
    }
}
```

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.use.type.headers=false

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order

spring.kafka.listener.concurrency=3
```

This concurrency property will be set
In ConcurrentKafkaListenerContainerFactory

Spring boot application startup

- KafkaAutoConfiguration — *Triggered by spring-kafka on class path*
  - Creates Bean: ConsumerFactory — *reads spring.kafka.consumer.* props, stores as Map*
  - Creates Bean: ConcurrentKafkaListenerContainerFactory — *holds reference of ConsumerFactory, no Kafka Consumer object created as of now.*
- @EnableKafka
  - KafkaListenerAnnotationBeanPostProcessor
    - Scan @KafkaListener
    - For each @KafkaListener, ask ConcurrentKafkaListenerContainerFactory to create bean — *Then for each KafkaListener, It will create N number of* KafkaListenerContainer and thus N number of KafkaConsumer
      - KafkaListenerContainer gets KafkaConsumer object from Consumer Factory
        - Poll loop starts